

METODO To String

El método `toString()` en Java es un método que se encuentra en la clase `Object`, la cual es la superclase de todas las clases en Java. El propósito de `toString()` es devolver una representación en forma de cadena de texto del objeto al que pertenece. Esta representación suele ser útil para la depuración y para obtener una descripción legible del objeto.

La implementación por defecto de `toString()` en la clase `Object` devuelve una cadena que incluye el nombre de la clase del objeto y el valor hexadecimal del código hash del objeto. Su firma es:

```
public String toString()
```

Aquí un ejemplo de su implementación por defecto:

```
public String toString() {  
    return getClass().getName() + "@" +  
    Integer.toHexString(hashCode());  
}
```

Sin embargo, es común sobrescribir este método en tus propias clases para proporcionar una representación más útil y legible del objeto. Por ejemplo:

```
public class Persona {  
    private String nombre;  
    private int edad;  
  
    public Persona(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
  
    @Override  
    public String toString() {  
        return "Persona[nombre=" + nombre + ", edad=" + edad + "]";  
    }  
}
```

En este ejemplo, la clase `Persona` sobrescribe el método `toString()` para devolver una cadena que contiene el nombre y la edad de la persona. De esta forma, cuando se imprima un objeto `Persona`, la salida será más significativa:

```
Persona persona = new Persona("Juan", 30);  
System.out.println(persona); // Imprime: Persona[nombre=Juan, edad=30]
```

Sobrescribir `toString()` es una buena práctica cuando necesitas que tus objetos tengan una representación en forma de cadena más descriptiva y útil.

EN CASO DE HERENCIA

En el caso de la herencia en Java, el método `toString()` puede ser sobrescrito en las subclases para proporcionar una representación más específica de esos objetos. Cuando una subclase sobrescribe `toString()`, puede llamar al método `toString()` de su superclase usando `super.toString()` para incluir información de la clase base en la representación del objeto.

Aquí hay un ejemplo para ilustrar cómo se maneja el método `toString()` en una jerarquía de clases con herencia:

```
// Clase base
public class Animal {
    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }

    @Override
    public String toString() {
        return "Animal[nombre=" + nombre + "]";
    }
}

// Subclase
public class Perro extends Animal {
    private String raza;

    public Perro(String nombre, String raza) {
        super(nombre); // Llamada al constructor de la clase base
        this.raza = raza;
    }

    @Override
    public String toString() {
        return super.toString() + ", Perro[raza=" + raza + "]";
    }
}

// Ejemplo de uso
public class Main {
    public static void main(String[] args) {
        Animal animal = new Animal("Generic Animal");
        Perro perro = new Perro("Fido", "Labrador");

        System.out.println(animal); // Imprime: Animal[nombre=Generic
Animal]
```

```
        System.out.println(perro); // Imprime: Animal[nombre=Fido],
        Perro[raza=Labrador]
    }
}
```

En este ejemplo:

1. **Clase base `Animal`:**

- Tiene un campo `nombre`.
- Sobrescribe el método `toString()` para devolver una cadena con el nombre del animal.

2. **Subclase `Perro`:**

- Extiende `Animal` y agrega un campo `raza`.
- Sobrescribe el método `toString()`.
- Llama al método `toString()` de la clase base (`super.toString()`) para incluir la información de `Animal` en su propia representación de cadena.

Al ejecutar el método `toString()` en un objeto `Perro`, se obtiene una cadena que incluye tanto la información de `Animal` como la información específica de `Perro`.

Este enfoque permite que las subclases enriquezcan la representación de cadena de los objetos heredados, manteniendo la información básica proporcionada por la clase base y añadiendo detalles específicos de la subclase.

EL PROBLEMA DE CLASES ASOCIADAS

Si tienes dos clases asociadas como `Persona` y `Domicilio` y cada una tiene una referencia a la otra, puedes encontrarte con problemas de recursividad infinita al sobrescribir el método `toString()` si no se maneja correctamente. Esto puede ocurrir porque el método `toString()` de `Persona` puede llamar al método `toString()` de `Domicilio`, y viceversa, creando un ciclo sin fin.

Para ilustrarlo, considera el siguiente ejemplo simplificado:

```
public class Domicilio {
    private String calle;
    private String ciudad;
    private Persona persona;

    public Domicilio(String calle, String ciudad, Persona persona) {
        this.calle = calle;
        this.ciudad = ciudad;
        this.persona = persona;
    }

    @Override
```

```
        public String toString() {
            return "Domicilio[calle=" + calle + ", ciudad=" + ciudad + ",
persona=" + persona + "]\n";
        }
    }

    public class Persona {
        private String nombre;
        private int edad;
        private Domicilio domicilio;

        public Persona(String nombre, int edad, Domicilio domicilio) {
            this.nombre = nombre;
            this.edad = edad;
            this.domicilio = domicilio;
        }

        @Override
        public String toString() {
            return "Persona[nombre=" + nombre + ", edad=" + edad + ",
domicilio=" + domicilio + "]\n";
        }
    }
}
```

En este caso, si intentas imprimir una instancia de `Persona`, esto llevará a un `StackOverflowError` debido a la recursión infinita entre `toString()` de `Persona` y `Domicilio`.

Para evitar este problema, puedes modificar el método `toString()` para no incluir la referencia completa de la otra clase. Aquí tienes una posible solución:

```
java
Copiar código
public class Domicilio {
    private String calle;
    private String ciudad;
    private Persona persona;

    public Domicilio(String calle, String ciudad, Persona persona) {
        this.calle = calle;
        this.ciudad = ciudad;
        this.persona = persona;
    }

    @Override
    public String toString() {
        // Evitar la recursión infinita no incluyendo a 'persona'
        directamente
        return "Domicilio[calle=" + calle + ", ciudad=" + ciudad +
"\n";
    }
}

public class Persona {
    private String nombre;
```

```
private int edad;
private Domicilio domicilio;

public Persona(String nombre, int edad, Domicilio domicilio) {
    this.nombre = nombre;
    this.edad = edad;
    this.domicilio = domicilio;
}

@Override
public String toString() {
    return "Persona[nombre=" + nombre + ", edad=" + edad + ",
domicilio=" + domicilio + "]\n";
}
}
```

Otra opción es usar una bandera temporal para controlar si ya se está generando el `toString()` de un objeto, lo que evitaría la recursión infinita, pero esto puede complicar el código y no es una práctica común:

```
public class Domicilio {
    private String calle;
    private String ciudad;
    private Persona persona;

    public Domicilio(String calle, String ciudad, Persona persona) {
        this.calle = calle;
        this.ciudad = ciudad;
        this.persona = persona;
    }

    @Override
    public String toString() {
        return "Domicilio[calle=" + calle + ", ciudad=" + ciudad +
"\n";
    }
}

public class Persona {
    private String nombre;
    private int edad;
    private Domicilio domicilio;
    private transient boolean inToString = false;

    public Persona(String nombre, int edad, Domicilio domicilio) {
        this.nombre = nombre;
        this.edad = edad;
        this.domicilio = domicilio;
    }

    @Override
    public String toString() {
        if (inToString) {
            return "Persona[nombre=" + nombre + ", edad=" + edad +
"\n";
        }
    }
}
```

```
        inToString = true;
        String result = "Persona[nombre=" + nombre + ", edad=" + edad
+ ", domicilio=" + domicilio + "];";
        inToString = false;
        return result;
    }
}
```

En resumen, debes ser cuidadoso al sobrescribir el método `toString()` en clases con referencias cíclicas. Una forma común y sencilla de evitar problemas es no incluir la referencia completa al objeto asociado en el método `toString()`, o incluir solo información relevante sin caer en la recursividad infinita.